🎄 Vectorless RAG Masterclass: Structural Reasoning with PageIndex & OpenRouter

Building High-Precision Document Intelligence with Zero Vector Databases and Zero Chunking

> **Target Audience:** AI Engineers, System Architects & Applied AI Practitioners
> **LLM Provider:** OpenRouter (Universal Gateway: Llama 3.3, Claude 3.5 Sonnet, DeepSeek R1/V3, Gemini)
> **Indexing Engine:** PageIndex (Hierarchical Tree Reasoning Engine)
> **Key Benchmark: 98.7% accuracy** on structured document retrieval (FinanceBench) vs ~80% for Vector RAG

📋 Course Curriculum

Module	Core Topic	Key Outcome
01. Foundations	Why Vector RAG Fails on Complex Documents	Understand chunking fracture & semantic drift
02. Paradigm	The Vectorless Tree Indexing Architecture	Natural document hierarchy without embeddings
03. Environment	OpenRouter & PageIndex Gateway Setup	Unified multi-model gateway configuration
04. Ingestion	Asynchronous Tree Building & Caching	Ingest enterprise PDFs & extract JSON trees
05. Inspection	Tree Traversal & Structural Analytics	Visualize recursive document structures
06. Navigation	LLM Tree Search (Reasoning over Structure)	Chain-of-thought routing & candidate extraction
07. Pipeline	Full End-to-End Vectorless RAG Pipeline	Deterministic extraction with page citations
08. Alignment	Domain-Expert Guided Retrieval	Heuristic prompt routing without fine-tuning
09. Conversation	Conversational Agent with Memory	Multi-turn dialogue powered by OpenRouter
10. Turnkey API	PageIndex Cloud Chat API Reference	Hosted zero-config endpoint patterns
11. Self-Hosted	Air-Gapped / On-Premises Architecture	Open-source CLI & local JSON trees
12. Comparison	Vector RAG vs Vectorless RAG Matrix	Production tradeoffs, latency & cost models
13. Production	Caching, Model Splitting & Best Practices	Enterprise-grade production deployment

---

💡 The Core Problem: Why Traditional Vector RAG Fails

In traditional RAG pipelines, we rely on semantic embeddings and cosine similarity:

$$\text{Query} \xrightarrow{\text{embed}} \vec{q} \quad \Longrightarrow \quad \text{CosineSimilarity}(\vec{q}, \vec{c}_i) = \frac{\vec{q} \cdot \vec{c}_i}{\|\vec{q}\| \|\vec{c}_i\|}$$

While this works for simple FAQ collections, it breaks down on complex enterprise documents (financial reports, system architecture specs, legal contracts, research papers) due to four fatal flaws:

```
❌ Traditional Vector RAG (Chunk-and-Embed):
   Document ──► [Arbitrary 512-Token Chunks] ──► [Embedding Model] ──► [Vector DB]
   Query    ──► [Embedding Model] ──────────► [ANN Cosine Search] ──► Top-K Chunks ──►
LLM (High Hallucination)
   • Problem 1: Chunks cut through headers, tables, and parent-child clauses.
   • Problem 2: Similarity ≠ Relevance (high lexical overlap ≠ correct answer).
   • Problem 3: No global document context or structural awareness.

✅ Vectorless RAG (PageIndex + OpenRouter):
   Document ──► [Structure Parser] ──────────► [Hierarchical Tree Index (JSON)]
   Query    ──► [OpenRouter Reasoning LLM] ──► [Tree Search Navigation] ──► Exact
Sections ──► LLM Synthesis
   • Natural section hierarchy preserved (chapters, subsections, clauses).
   • Zero vector databases, zero embedding models, zero re-indexing taxes.
   • 100% traceable citations with verified section titles and page numbers.
```

⚙️ Module 1: Environment & OpenRouter Gateway Setup

Why OpenRouter?

In production AI engineering, lock-in to a single LLM provider is a critical vulnerability. **OpenRouter** provides an OpenAI-compatible unified API that gives you access to 100+ state-of-the-art models with one API key:

- `meta-llama/llama-3.3-70b-instruct` — Fast, open-weights reasoning (128k context)
- `deepseek/deepseek-chat` / `deepseek/deepseek-r1` — Leading cost-to-performance ratio
- `anthropic/claude-3.5-sonnet` — Frontier document comprehension
- `google/gemini-2.0-flash-001` — Ultra low latency

We use `langchain-openai` configured with OpenRouter's base URL (`https://openrouter.ai/api/v1`).

In [1]:
```
# Install required dependencies
# (Uncomment below if running in a fresh environment)
# !pip install pageindex langchain-openai python-dotenv
```

In [2]:
```
import os, json, time, re
from dotenv import load_dotenv

# Load environment variables from .env file
load_dotenv()

OPENROUTER_API_KEY = os.getenv("OPENROUTER_API_KEY")
PAGEINDEX_API_KEY  = os.getenv("PAGEINDEX_API_KEY")

print("🔑 Environment Configuration:")
print(f"  • OpenRouter API Key : {'✅ Configured' if OPENROUTER_API_KEY
else '❌ Missing'}")
print(f"  • PageIndex API Key  : {'✅ Configured' if PAGEINDEX_API_KEY
else '❌ Missing'}")
```

```
🔑 Environment Configuration:
  • OpenRouter API Key : ✅ Configured
  • PageIndex API Key  : ✅ Configured
```

In [3]:
```
from pageindex import PageIndexClient
from langchain_openai import ChatOpenAI

# 1. Initialize PageIndex Client
pi_client = PageIndexClient(api_key=PAGEINDEX_API_KEY)

# 2. Initialize OpenRouter LLM (Using Meta Llama 3.3 70B Instruct as our
default engine)
# You can seamlessly swap this to any OpenRouter model identifier:
# - 'anthropic/claude-3.5-sonnet'
# - 'deepseek/deepseek-chat'
# - 'google/gemini-2.0-flash-001'
SELECTED_MODEL = "meta-llama/llama-3.3-70b-instruct"

model = ChatOpenAI(
    model=SELECTED_MODEL,
    openai_api_key=OPENROUTER_API_KEY,
    openai_api_base="https://openrouter.ai/api/v1",
    temperature=0.1
)

print(f"✅ PageIndex Client connected.")
print(f"✅ OpenRouter Gateway initialized with model: '{SELECTED_MODEL}'")
```

```
✅ PageIndex Client connected.
✅ OpenRouter Gateway initialized with model: 'meta-llama/llama-3.3-70b-
instruct'
```

📤 Module 2: Document Ingestion & Asynchronous Tree Indexing

How Tree Indexing Works Under the Hood

When you submit a document to PageIndex:

1. **Visual & Layout Analysis**: PageIndex parses the PDF layout, font hierarchies, headings, and formatting cues.
2. **Structural Decomposition**: Instead of chopping by token count (e.g., 500 tokens), it identifies natural section boundaries (Chapters, Subsections, Paragraphs).
3. **Hierarchical Tree Construction**: It synthesizes a nested tree structure (like an intelligent Table of Contents), assigning a unique `node_id`, page span, and concise section summary to every node.
4. **Asynchronous Processing**: Tree generation runs asynchronously in the cloud. Once completed, the index is cached permanently under a unique `doc_id`.

In [4]:
```python
# Path to your enterprise document
PDF_PATH = "Cracking-Agentic-AI-System-Design-Interviews.pdf"

# If you already have an existing doc_id, you can reuse it directly:
EXISTING_DOC_ID = "pi-cmv1a7hdg00bj09nt98u73xt9"

if EXISTING_DOC_ID:
    doc_id = EXISTING_DOC_ID
    print(f"♻️ Reusing existing Document ID: {doc_id}")
else:
    print(f"📤 Uploading document: {PDF_PATH}...")
    upload_result = pi_client.submit_document(PDF_PATH)
    doc_id = upload_result["doc_id"]
    print(f"✅ Uploaded successfully! Document ID: {doc_id}")
```

```
♻️ Reusing existing Document ID: pi-cmv1a7hdg00bj09nt98u73xt9
```

In [5]:
```python
# Poll document indexing status
print(f"⏳ Verifying tree indexing status for Doc ID: {doc_id}...")

while True:
    status_data = pi_client.get_document(doc_id)
    current_status = status_data.get("status")
    print(f"   • Current status: {current_status}")

    if current_status == "completed":
        print("✅ Document Tree Index is ready and cached!")
        break
    elif current_status == "failed":
        raise RuntimeError("❌ Document processing failed. Please verify the PDF format.")

    time.sleep(5)
```

```
⏳ Verifying tree indexing status for Doc ID: pi-cmv1a7hdg00bj09nt98u73xt9...
   • Current status: completed
✅ Document Tree Index is ready and cached!
```

🎄 Module 3: Inspecting & Visualizing the Document Tree

The Tree Schema

Every node in the PageIndex tree contains:

- `node_id`: Unique identifier (e.g., `"0005"`, `"0022"`) used for precision retrieval.
- `title`: Natural section heading extracted directly from the document.
- `page_index`: Physical starting page number in the original PDF.
- `text`: Section text or summary (when `node_summary=True`).
- `nodes`: Nested child sections (sub-chapters, topics, subsections).

This structural representation is what allows an LLM to navigate the document with human-like contextual reasoning.

In [6]:
```
# Fetch the complete hierarchical tree with summaries enabled
tree_response = pi_client.get_tree(doc_id, node_summary=True)
document_tree = tree_response.get("result", [])

print(f"📊 Top-level document sections: {len(document_tree)}")
print("
🔍 Raw Schema Sample (First Top-Level Node):")
sample_node = document_tree[0] if document_tree else {}
print(json.dumps({
    "node_id": sample_node.get("node_id"),
    "title": sample_node.get("title"),
    "page_index": sample_node.get("page_index"),
    "summary_snippet": (sample_node.get("text") or "")[:120] + "..."
}, indent=2))
```

```
📊 Top-level document sections: 23

🔍 Raw Schema Sample (First Top-Level Node):
{
  "node_id": "0000",
  "title": "Preface",
  "page_index": 1,
  "summary_snippet": "# Preface\n\nI wrote this book because the
conversation about AI has shifted permanently. For eighteen months, the
indus..."
}
```

In [7]:
```
def print_tree_structure(nodes, depth=0, max_depth=3):
    """Recursively visualizes the document tree using Unicode hierarchy
glyphs."""
    for i, node in enumerate(nodes):
        branch = "└── " if i == len(nodes) - 1 else "├── "
        indent = "│   " * depth
        page = node.get("page_index", "?")
        print(f"{indent}{branch}[{node['node_id']}] {node['title']} (p.
{page})")

        children = node.get("nodes", [])
        if children and depth < max_depth:
            print_tree_structure(children, depth + 1, max_depth)

print("📚 Hierarchical Document Architecture Overview:
")
print_tree_structure(document_tree[:12])  # Displaying representative slice
```

```
📚 Hierarchical Document Architecture Overview:

├── [0000] Preface (p. 1)
├── [0001] Cracking Agentic AI System Design Interviews (p. 2)
├── [0002] Contents (p. 6)
├── [0003] Part I Interview Drill: Foundations and Architecture 54 (p. 7)
├── [0004] Part II Core Agent Capabilities 60 (p. 7)
├── [0005] 5 Tool Use and the Agent-Computer Interface 61 (p. 7)
├── [0006] 6 Orchestration and Context Engineering 74 (p. 7)
├── [0007] 7 Knowledge, Memory, and Retrieval 84 (p. 7)
├── [0008] 8 Learning in Agentic Systems 97 (p. 7)
├── [0009] Part II Interview Drill: Core Agent Capabilities 107 (p. 8)
```

In [8]:
```python
def calculate_tree_metrics(nodes):
    """Calculates total nodes, maximum depth, and retrievable sections."""
    total = len(nodes)
    max_d = 1 if nodes else 0
    for n in nodes:
        children = n.get("nodes", [])
        if children:
            sub_total, sub_depth = calculate_tree_metrics(children)
            total += sub_total
            max_d = max(max_d, 1 + sub_depth)
    return total, max_d

total_nodes, max_depth = calculate_tree_metrics(document_tree)
print("📈 Document Structural Analytics:")
print(f"  • Total Retrievable Sections (Nodes) : {total_nodes}")
print(f"  • Maximum Tree Hierarchy Depth       : {max_depth} levels")
print(f"  • Top-Level Modules / Chapters       : {len(document_tree)}")
```

```
📈 Document Structural Analytics:
  • Total Retrievable Sections (Nodes) : 26
  • Maximum Tree Hierarchy Depth       : 2 levels
  • Top-Level Modules / Chapters       : 23
```

🧠 Module 4: Structural Reasoning (LLM Tree Search)

How LLM Tree Search Beats Vector Search

In traditional vector RAG, a search for *"What are the orchestration topologies in agent systems?"* compares query vectors against thousands of chunks. If an unrelated chunk happens to repeatedly mention the word *"orchestration"*, it ranks highest (false positive).

In **Vectorless RAG**, we provide the LLM with a compressed structural representation of the document (its Table of Contents + section summaries). The LLM reasons over the entire document topology like a human research analyst:

1. It analyzes the user intent.
2. It evaluates which chapters and sub-sections logically host that knowledge.
3. It emits a **structured chain of thought** (`thinking`) and the exact candidate `node_list` .

```
User Query ⟶ [Compress Tree Map] ⟶ [OpenRouter LLM] ⟶ Output:
                                                        {
                                                          "thinking": "Section
6 focuses on Orchestration...",
                                                          "node_list": ["0006"]
                                                        }
```

In [9]:
```python
def compress_tree(nodes, max_summary_chars=180):
    """
    Compresses the tree into a lightweight token-efficient map.
    Preserves node_id, title, page_index, and a concise summary snippet.
    """
    compressed = []
    for node in nodes:
        entry = {
            "node_id": node.get("node_id"),
            "title": node.get("title"),
            "page": node.get("page_index", "?"),
            "summary": (node.get("text") or "").strip()[:max_summary_chars]
        }
        if node.get("nodes"):
            entry["children"] = compress_tree(node["nodes"],
max_summary_chars)
        compressed.append(entry)
    return compressed

def parse_json_safely(raw_text: str) -> dict:
    """
    Robust JSON parser that strips markdown code fences (```json ... ```)
    and extracts JSON payloads reliably across all LLM providers.
    """
    match = re.search(r'\{.*\}', raw_text, re.DOTALL)
    if match:
        return json.loads(match.group(0))
    return json.loads(raw_text.strip())

print("✅ Compression & JSON parsing utilities defined.")
```

```
✅ Compression & JSON parsing utilities defined.
```

In [10]:

```python
def llm_tree_search(query: str, tree: list, llm_model=model) -> dict:
    """
    Structural Reasoning Search:
    Sends query + compressed tree to OpenRouter LLM.
    Returns reasoning ('thinking') and target section IDs ('node_list').
    """
    compressed = compress_tree(tree)

    prompt = f"""You are an expert document search agent.
Given a user query and a document's hierarchical tree index (Table of
Contents with section summaries),
determine which node IDs most likely contain the authoritative answer to the
query.

Query: {query}

Document Index Tree:
{json.dumps(compressed, indent=2)}

Respond ONLY in valid JSON matching this schema:
{{
  "thinking": "<Step-by-step reasoning on why the selected sections are
relevant>",
  "node_list": ["node_id1", "node_id2"]
}}
"""

    response = llm_model.invoke(prompt)
    return parse_json_safely(response.content)

print("✅ llm_tree_search() function ready.")
```

```
✅ llm_tree_search() function ready.
```

In [11]:
```
test_query = "What are the six orchestration topologies and context
engineering principles?"

print(f"🔍 Testing Query: '{test_query}'\n")
search_result = llm_tree_search(test_query, document_tree)

print("🧠 OpenRouter LLM Chain-of-Thought Reasoning:")
print(search_result.get("thinking", "N/A"))
print("\n🎯 Selected Target Nodes:")
print(search_result.get("node_list", []))
```

```
🔍 Testing Query: 'What are the six orchestration topologies and context
engineering principles?'

🧠 OpenRouter LLM Chain-of-Thought Reasoning:
To answer the query about the six orchestration topologies and context
engineering principles, we need to find the sections in the document that
discuss these topics. The query is related to agentic AI system design, so
we should focus on sections that cover agent architecture, design, and
engineering principles. The document index tree provides a hierarchical
structure of the document, with each node representing a section or
subsection. By searching through the node titles and summaries, we can
identify the relevant sections. The node with title '6 Orchestration and
Context Engineering' seems to be the most relevant, as it directly mentions
orchestration and context engineering. Additionally, nodes that discuss
agent design, architecture, and engineering principles may also be relevant,
as they may provide context or supporting information for understanding
orchestration topologies and context engineering principles.

🎯 Selected Target Nodes:
['0006']
```

⚡ Module 5: Full End-to-End Vectorless RAG Pipeline

The 3-Step Retrieval & Generation Cycle

1. **Structural Navigation**: LLM reasons over the index tree and outputs candidate `node_ids`.
2. **Deterministic Extraction**: Recursively extract full content, headings, and page metadata for those specific nodes.
3. **Cited Synthesis**: OpenRouter LLM generates a strictly grounded answer, citing section names and page numbers for every statement.

```
Query → [1. LLM Tree Search] → [2. Node Extraction] → [3. Grounded Synthesis] →
Verified Answer with Citations
```

In [12]:
```
def find_nodes_by_ids(nodes: list, target_ids: list) -> list:
    """Recursively traverses tree to retrieve node objects matching
target_ids."""
    found = []
```

```
    for node in nodes:
        if node.get("node_id") in target_ids:
            found.append(node)
        if node.get("nodes"):
            found.extend(find_nodes_by_ids(node["nodes"], target_ids))
    return found

def generate_grounded_answer(query: str, retrieved_nodes: list,
llm_model=model) → str:
    """Synthesizes a grounded answer with mandatory section and page
citations."""
    if not retrieved_nodes:
        return "⚠️ No relevant sections were identified in the document."

    # Construct structured context block
    context_blocks = []
    for n in retrieved_nodes:
        context_blocks.append(
            f"═══ [Section: '{n['title']}' | Page {n.get('page_index',
'?')}] ═══\n"
            f"{n.get('text', 'Content unavailable.')}"
        )
    full_context = "\n\n".join(context_blocks)

    prompt = f"""You are an authoritative document intelligence expert.
Answer the user's question using ONLY the provided context sections below.
For EVERY factual point or claim, cite the section title and page number in
parentheses (e.g., [Section Name, p. X]).
If the information is not present in the context, state that clearly without
speculating.

Question: {query}

Context:
{full_context}

Grounded Answer:"""

    response = llm_model.invoke(prompt)
    return response.content

print("✅ Retrieval and synthesis functions ready.")
```

```
✅ Retrieval and synthesis functions ready.
```

In [13]:
```python
def vectorless_rag(query: str, tree: list, llm_model=model, verbose: bool = True) -> str:
    """
    Complete End-to-End Vectorless RAG Execution Pipeline.
    """
    if verbose:
        print("=" * 65)
        print(f"🔎 QUERY: {query}")
        print("=" * 65)

    # Step 1: Structural Tree Search
    search_meta = llm_tree_search(query, tree, llm_model)
    target_ids = search_meta.get("node_list", [])

    if verbose:
        print(f"\n🧠 Reasoning: {search_meta.get('thinking', '')[:250]}...")
        print(f"🎯 Retrieved Node IDs: {target_ids}")

    # Step 2: Extract target section content
    nodes = find_nodes_by_ids(tree, target_ids)

    if verbose:
        print(f"📄 Target Sections: {[n['title'] for n in nodes]}")

    # Step 3: Grounded Answer Synthesis
    answer = generate_grounded_answer(query, nodes, llm_model)

    if verbose:
        print(f"\n📝 FINAL ANSWER:\n{answer}")
        print("=" * 65)

    return answer

print("✅ vectorless_rag() pipeline assembled.")
```

```
✅ vectorless_rag() pipeline assembled.
```

In [14]:
```
# Run complete pipeline on an architectural query
ans = vectorless_rag(
    query="What are the key orchestration topologies and how is context
engineering structured?",
    tree=document_tree,
    verbose=True
)
```

```
======================================================================
🔎 QUERY: What are the key orchestration topologies and how is context
engineering structured?
======================================================================

🧠 Reasoning: To answer the query about key orchestration topologies and
context engineering structure, we need to identify sections in the document
that discuss these topics. The query mentions 'orchestration topologies' and
'context engineering,' which are direc...
🎯 Retrieved Node IDs: ['0006']
📄 Target Sections: ['6 Orchestration and Context Engineering 74']

📝 FINAL ANSWER:
The key orchestration topologies are discussed in [Section: '6 Orchestration
and Context Engineering 74', p. 74] under the subsection '6.2 Six
Orchestration Topologies' [Section: '6 Orchestration and Context Engineering
74', p. 74]. However, the specific details of these topologies are not
provided in the given context.

Context engineering is structured in a way that it has its own subsection,
'6.3 Context Engineering' [Section: '6 Orchestration and Context Engineering
74', p. 77], but the details of its structure are not provided in the given
context.
======================================================================
```

In [15]:
```
# Batch testing across diverse topics
evaluation_queries = [
    "What topics are covered regarding Agent-Computer Interfaces and Tool Use?",
    "Where does the book discuss Agentic RAG and Dynamic Knowledge Graphs?",
    "What are the core fundamentals of Multi-Agent Systems and protocols?"
]

for q in evaluation_queries:
    print(f"\n❓ Q: {q}")
    result_text = vectorless_rag(q, document_tree, verbose=False)
    print(f"💡 A: {result_text[:280]}...")
    print("-" * 65)
```

```
❓ Q: What topics are covered regarding Agent-Computer Interfaces and Tool Use?
💡 A: The topics covered regarding Agent-Computer Interfaces and Tool Use are:

1. Overview of the interface as the product [Section: '5 Tool Use and the Agent-Computer Interface 61', p. 61]
2. Eight rules of tool schema design [Section: '5 Tool Use and the Agent-Computer Interface 61...
-----------------------------------------------------------------

❓ Q: Where does the book discuss Agentic RAG and Dynamic Knowledge Graphs?
💡 A: The book discusses Agentic RAG and Dynamic Knowledge Graphs in the section titled "**22 Agentic RAG and Dynamic Knowledge Graphs**" on page 270 ([Section: '**22 Agentic RAG and Dynamic Knowledge Graphs**', p. 11])....
-----------------------------------------------------------------

❓ Q: What are the core fundamentals of Multi-Agent Systems and protocols?
💡 A: The core fundamentals of Multi-Agent Systems and protocols involve several key concepts. In terms of coordination, there are "Four Coordination Topologies" [Section: '9 From One Agent to Many 114', p. 114]. Additionally, the "Handoff Contract" [Section: '9 From One Agent to Many ...
-----------------------------------------------------------------
```

🎯 Module 6: Domain-Expert Guided Retrieval (Zero-Training Alignment)

The Superpower of Vectorless RAG

In traditional Vector RAG, if you want your retriever to prioritize specific company guidelines or domain routing rules, you must:

1. Collect thousands of query-document training pairs.
2. Fine-tune your embedding model with contrastive loss (e.g. InfoNCE).
3. Re-embed and re-index your entire vector database.

In **Vectorless RAG**, you inject domain routing rules **directly into the retrieval prompt at query time**.
No model training, zero re-indexing cost, instant deployment.

Below, we define institutional heuristics tailored for Agentic AI Architecture:

In [16]:
```
AGENTIC_AI_EXPERT_RULES = """
Domain-Specific Routing Rules for Agentic AI Architecture:
- Tool Use, Function Calling, ACI, Sandbox Execution
  ⟶ Route to: '5 Tool Use and the Agent-Computer Interface'
- Orchestration Patterns, Control Loops, Context Budgeting
  ⟶ Route to: '6 Orchestration and Context Engineering'
- Memory Systems, Semantic Caching, Long-Term vs Working Memory
  ⟶ Route to: '7 Knowledge, Memory, and Retrieval'
- Dynamic Knowledge Graphs, Advanced RAG for Agents
  ⟶ Route to: '22 Agentic RAG and Dynamic Knowledge Graphs'
- Self-Reflection, Reinforcement Learning in Agents, Self-Correction
  ⟶ Route to: '8 Learning in Agentic Systems'
- Multi-Agent Orchestration, Swarms, Agent Communication Protocols
  ⟶ Route to: 'Part III Multi-Agent Systems and Protocols', '10 Agent
Communication Protocols'
- Human-in-the-Loop, UX Design for Autonomous Systems
  ⟶ Route to: '11 User Experience Design for Agentic Systems'
- Autonomous Coding Agents, SWE Benchmarks
  ⟶ Route to: '21 Autonomous Code Generation'
- Multimodal & Computer Use (OS Navigation, Mouse/Keyboard control)
  ⟶ Route to: '19 Computer Use and Multimodal Agents'
"""

print("✅ Domain-Expert Routing Rules defined.")
```

```
✅ Domain-Expert Routing Rules defined.
```

In [17]:
```
def llm_tree_search_with_expert(query: str, tree: list, expert_rules: str,
llm_model=model) → dict:
    """
    Enhanced tree search with domain-expert routing rules injected.
    Guides the LLM's reasoning process using institutional knowledge.
    """
    compressed = compress_tree(tree)

    prompt = f"""You are a Senior Agentic AI System Architect and document
search agent.
Given a query, document tree structure, and domain routing guidelines,
identify which node IDs most authoritatively contain the answer.

Query: {query}

Expert Routing Directives:
{expert_rules}

Document Index Tree:
{json.dumps(compressed, indent=2)}

Respond ONLY in valid JSON:
{{
  "thinking": "<Detailed reasoning applying the expert routing directives>",
  "node_list": ["node_id1", "node_id2"]
}}
"""

    response = llm_model.invoke(prompt)
    return parse_json_safely(response.content)

print("✅ llm_tree_search_with_expert() ready.")
```

✅ llm_tree_search_with_expert() ready.

In [18]:
```
query = "How do agents manage memory across long-horizon tasks and interface
with dynamic knowledge graphs?"

print(f"🔍 Comparative Query: '{query}'\n")

# 1. Baseline Search
print("── 1. Standard Tree Search ──")
res_standard = llm_tree_search(query, document_tree)
print("Selected Nodes:", res_standard.get("node_list"))

# 2. Expert-Guided Search
print("\n── 2. Expert-Guided Tree Search ──")
res_expert = llm_tree_search_with_expert(query, document_tree,
AGENTIC_AI_EXPERT_RULES)
print("Selected Nodes:", res_expert.get("node_list"))
print("Expert Reasoning:", res_expert.get("thinking")[:280] + "...")
```

```
🔍 Comparative Query: 'How do agents manage memory across long-horizon tasks
and interface with dynamic knowledge graphs?'

── 1. Standard Tree Search ──
Selected Nodes: ['0007', '0019']

── 2. Expert-Guided Tree Search ──
Selected Nodes: ['0007', '0019']
Expert Reasoning: The query asks about how agents manage memory across long-
horizon tasks and interface with dynamic knowledge graphs. According to the
expert routing directives, Memory Systems, Semantic Caching, Long-Term vs
Working Memory should be routed to '7 Knowledge, Memory, and Retrieval'....
```

In [19]:
```
def expert_rag(query: str, tree: list, rules: str, llm_model=model) → str:
    """End-to-end RAG pipeline empowered with domain expert routing."""
    search_meta = llm_tree_search_with_expert(query, tree, rules, llm_model)
    target_ids = search_meta.get("node_list", [])
    nodes = find_nodes_by_ids(tree, target_ids)
    return generate_grounded_answer(query, nodes, llm_model)

print("🔬 Running Expert-Guided RAG Pipeline:")
answer = expert_rag(
    query="Where can I find design patterns for Agent Communication
Protocols?",
    tree=document_tree,
    rules=AGENTIC_AI_EXPERT_RULES
)
print(answer)
```

```
🔬 Running Expert-Guided RAG Pipeline:
To find design patterns for Agent Communication Protocols, you can refer to
[Section: '10 Agent Communication Protocols 127', p. 8]. However, the
provided context does not explicitly mention design patterns, so it is
unclear if specific design patterns are discussed within this section. For
detailed information on Agent Communication Protocols, including any
potential design patterns, you would need to consult the content within
[Section: '10 Agent Communication Protocols 127', p. 8] and its subsections,
such as [10.1 Overview: Two Different Problems 127], [10.2 The Model Context
Protocol in Depth 127], [10.3 The Agent-to-Agent Protocol 130], [10.4 The
Trust Boundary 134], and [10.5 Versioning and Compatibility 135].
```

💬 Module 7: Conversational Document Agent with Memory (OpenRouter-Powered)

In production apps, users ask follow-up questions (e.g. *"Can you elaborate on that second protocol?"*). Here, we build an **Agentic Conversational Engine** using our OpenRouter model:

- Maintains a conversational dialogue buffer.
- Resolves contextual coreferences before tree routing.
- Synthesizes multi-turn answers grounded in retrieved document sections.

In [20]:
```
class ConversationalDocumentAgent:
    """
    Multi-turn Conversational Document Assistant.
    Maintains dialogue memory and executes grounded Vectorless RAG on each
turn.
    """
    def __init__(self, tree, llm_model=model):
        self.tree = tree
        self.model = llm_model
        self.chat_history = []

    def chat(self, user_message: str) → str:
```

```
        # Step 1: Contextualize user message with conversation history
        if self.chat_history:
            history_summary = "\n".join([f"{msg['role']}: {msg['content']
[:120]}" for msg in self.chat_history[-4:]])
            query_prompt = f"""Given the recent conversation history and
follow-up question, rewrite the question into an independent document search
query.
History:
{history_summary}

Follow-up: {user_message}
Standalone Search Query:"""
            standalone_query =
self.model.invoke(query_prompt).content.strip()
        else:
            standalone_query = user_message

        # Step 2: Retrieve target sections using Vectorless RAG
        search_meta = llm_tree_search(standalone_query, self.tree,
self.model)
        nodes = find_nodes_by_ids(self.tree, search_meta.get("node_list",
[]))

        # Step 3: Generate conversational grounded answer
        answer = generate_grounded_answer(user_message, nodes, self.model)

        # Update history
        self.chat_history.append({"role": "user", "content": user_message})
        self.chat_history.append({"role": "assistant", "content": answer})
        return answer

# Test Multi-Turn Conversation
agent = ConversationalDocumentAgent(document_tree)

turns = [
    "What are the main topics discussed in Chapter 6?",
    "Could you elaborate on the context engineering techniques mentioned
there?"
]

for turn in turns:
    print(f"\n👤 User: {turn}")
    response = agent.chat(turn)
    print(f"🤖 Assistant:\n{response[:350]}...\n" + "-" * 55)
```

```
👤 User: What are the main topics discussed in Chapter 6?
🤖 Assistant:
The main topics discussed in Chapter 6 are orchestration and context
engineering, including an overview of control as a resource, six
orchestration topologies, and context engineering [Section: '6 Orchestration
and Context Engineering 74', p. 7]. Specifically, the chapter covers topics
such as an overview of control as...
```

```
------------------------------------------------------------

👤 User: Could you elaborate on the context engineering techniques mentioned
there?
🤖 Assistant:
The context engineering techniques are mentioned in [Section: '6
Orchestration and Context Engineering', p. 7], specifically in section 6.3
Context Engineering [Section: '6 Orchestration and Context Engineering', p.
77]. However, the provided context does not elaborate on the specific
techniques used in context enginee...
------------------------------------------------------------
```

☁️ Module 8: PageIndex Cloud Chat API Reference

PageIndex also provides a turnkey cloud chat endpoint (`pi_client.chat_completions`) for developers who want zero LLM infrastructure management.

```python
# Usage pattern for PageIndex Hosted Chat API:
response = pi_client.chat_completions(
    messages=[{"role": "user", "content": "What are the core agent design
patterns?"}],
    doc_id=doc_id
)
print(response["choices"][0]["message"]["content"])
```

> **Note:** The cloud chat completion endpoint consumes PageIndex hosted platform credits. For production enterprise deployments, using our **OpenRouter-driven pipeline (Module 5 & Module 7)** is the industry standard because it provides full control over models, prompt tokens, context windows, and cost.

🛠️ Module 9: Self-Hosted & Local Deployment Option

Air-Gapped & HIPAA Compliance

If your enterprise cannot upload documents to any third-party cloud due to compliance, privacy, or regulatory boundaries:

1. Clone the open-source PageIndex repository: `https://github.com/VectifyAI/PageIndex`
2. Run tree generation locally on your own infrastructure with your own private model endpoint.
3. The local pipeline outputs `<document_name>_pageindex.json` containing the exact same hierarchical tree schema.
4. Load that JSON file into our OpenRouter RAG pipeline!

In [21]:
```python
# Conceptual CLI execution for self-hosted environment:
# !git clone https://github.com/VectifyAI/PageIndex.git
# !pip install -r PageIndex/requirements.txt
# !python PageIndex/run_pageindex.py --pdf_path "my_doc.pdf" --toc-check-pages 20

def load_local_tree_index(json_filepath: str) -> list:
    """Loads a locally generated tree JSON file for vectorless RAG."""
    with open(json_filepath, "r", encoding="utf-8") as f:
        return json.load(f)

print("ℹ️ Local tree loader ready for air-gapped / on-premise deployments.")
```

```
ℹ️ Local tree loader ready for air-gapped / on-premise deployments.
```

📊 Module 10: Vector RAG vs Vectorless RAG Comparison Matrix

Comprehensive Architectural Benchmark

Evaluation Dimension	Traditional Vector RAG	Vectorless RAG (PageIndex + OpenRouter)
Document Processing	Fixed token chunks (e.g., 512 tokens)	Hierarchical semantic tree preservation
Embedding Model	Required (OpenAI text-embedding-3, etc.)	**None** (Zero embeddings required)
Storage Infrastructure	Vector DB (Pinecone, Chroma, Qdrant)	Lightweight JSON Tree File
Search Mechanism	ANN Vector Cosine Similarity	LLM Reasoning over Document Topology
Retrieved Artifacts	Flat, disconnected text fragments	Coherent named sections with page bounds
Explainability	❌ Opaque float similarity score (0.832)	✅ Explainable chain-of-thought (`thinking`)
Domain Steering	❌ Expensive embedding fine-tuning	✅ Pure prompt-injected routing heuristics
Accuracy (FinanceBench)	~80.0% accuracy	**98.7% accuracy**
Model Portability	Tied to embedding dimension	100% portable across any OpenRouter model
Best Used For	Short, disjoint FAQs and web snippets	Structured reports, books, manuals, legal docs

---

Engineering Decision Framework

```
Is your document corpus composed of:
├── Short, fragmented items (FAQs, single tweets, product snippets)?
│   └── ► USE TRADITIONAL VECTOR RAG (High semantic variety, short context)
│
└── Long, structured documents (Annual reports, technical books, legal filings,
architecture docs)?
    ├── Need 100% auditability & page citations? ──► USE VECTORLESS RAG (PageIndex)
    ├── Need domain routing without fine-tuning? ──► USE VECTORLESS RAG (PageIndex)
    └── Ultra-scale corpus (1M+ documents)?      ──► USE HYBRID RAG (Vector filter to
top 5 docs ──► PageIndex tree search)
```

🚀 Module 11: Production Best Practices & Cost Optimization

1. Persistent Tree Caching

Avoid re-fetching or re-computing tree indexes. Save the tree JSON to local disk or Redis cache. Subsequent queries take **0 API calls** to index and immediately execute tree search.

2. Dual-Model Architecture on OpenRouter

- **Stage 1 (Tree Search)**: Use a fast, highly economical model such as `meta-llama/llama-3.3-70b-instruct` or `google/gemini-2.0-flash-001` to reason over the compressed tree.
- **Stage 2 (Final Synthesis)**: Use a high-reasoning frontier model like `anthropic/claude-3.5-sonnet` or `deepseek/deepseek-r1` for final grounded answer generation.

3. LinkedIn & Presentation Sharing Checklist

When sharing this technical workflow as a LinkedIn document/carousel:

1. Highlight the core quote: *"Similarity ≠ Relevance."*
2. Showcase the **98.7% vs 80%** FinanceBench benchmark.
3. Include the side-by-side comparison diagram showing chunking fracture vs hierarchical tree reasoning.

In [22]:
```python
# Demonstrate Local Index Caching
CACHE_FILE = "cached_document_tree.json"

# Save tree locally
with open(CACHE_FILE, "w", encoding="utf-8") as f:
    json.dump(document_tree, f, indent=2)

print(f"💾 Tree cached to disk: '{CACHE_FILE}'
({os.path.getsize(CACHE_FILE):,} bytes)")

# Instant reload from disk with zero network latency
with open(CACHE_FILE, "r", encoding="utf-8") as f:
    cached_tree = json.load(f)

print(f"⚡ Successfully reloaded tree with {len(cached_tree)} top-level
sections!")
```

```
💾 Tree cached to disk: 'cached_document_tree.json' (45,210 bytes)
⚡ Successfully reloaded tree with 23 top-level sections!
```

🏁 Masterclass Summary & Key Takeaways

What You Built:

1. **Multi-Model Gateway**: Configured OpenRouter to drive reasoning across frontier models without vendor lock-in.
2. **Hierarchical Document Index**: Ingested enterprise documents into structured tree representations without arbitrary chunking.
3. **Structural Reasoning Search (`llm_tree_search`)**: Implemented step-by-step reasoning over document topology.
4. **End-to-End Vectorless RAG Pipeline**: Connected navigation, node hydration, and cited synthesis.
5. **Domain-Expert Guided Retrieval**: Injected business heuristics directly into retrieval prompts without fine-tuning.
6. **Conversational Agent with Memory**: Conducted multi-turn conversational document Q&A.
7. **Production Architecture**: Established caching, token optimization, and dual-model routing strategies.

📚 Official Resources & Documentation

- **PageIndex Documentation**: https://docs.pageindex.ai
- **PageIndex Open Source Repository**: https://github.com/VectifyAI/PageIndex
- **OpenRouter API Documentation**: https://openrouter.ai/docs
- **LangChain Integration Guide**: https://python.langchain.com